

Análise e Extração de Padrões em Dados Textuais Desestruturados: Uma Abordagem Prática Utilizando Expressões Regulares

Daniel A. Sanchez¹, Dhario N. Evangelista¹, Henrique M. P. Silva¹, Khey Niwa¹,
João D. C. de Souza¹

¹Instituto de Ciências Exatas e Naturais – Universidade Federal do Pará (UFPA) Belém
– PA – Brasil

63amorimsanchezdaniel@gmail.com, dhario.evangelista@icen.ufpa.br,
henriquemascarenhas557@gmail.com, niwakhey@gmail.com,
jd3870894@gmail.com

Abstract. *This paper describes the development of a Python computational engine focused on extracting and validating patterns in unstructured textual databases. Using Regular Expressions (Regex), the solution adopts a two-stage architecture: a permissive search (re.finditer) followed by strict validation (re.match). The object-oriented system empirically processed 38,672 occurrences distributed across four real databases (logs, chats, CSVs, and free texts). The results show high efficiency in mitigating false positives, accurately isolating 3,377 malformed strings and ensuring robustness in data sanitization.*

Resumo. *Este artigo descreve o desenvolvimento de um motor computacional em Python focado na extração e validação de padrões em bases de dados textuais desestruturadas. Utilizando Expressões Regulares (Regex), a solução adota uma arquitetura de duplo estágio: uma busca permissiva (re.finditer) seguida de validação estrita (re.match). O sistema orientado a objetos processou empiricamente 38.672 ocorrências distribuídas em quatro bases reais (logs, chats, CSVs e textos livres). Os resultados evidenciam alta eficiência na mitigação de falsos positivos, isolando com precisão 3.377 cadeias malformadas e garantindo robustez na higienização de dados.*

1. Introdução

No cenário computacional contemporâneo, a onipresença de sistemas digitais gera um fluxo contínuo e massivo de dados operacionais. Contudo, uma parcela significativa dessas informações textuais encontra-se desestruturada, caracterizando-se pela presença de ruídos, inconsistências de codificação e múltiplos formatos de representação simultâneos. Registros de logs de servidores, exportações brutas de bancos de dados, históricos de aplicações de mensageria instantânea e cadastros operacionais frequentemente carecem de uma padronização sintática rigorosa. O tratamento manual ou puramente linear de tais volumes de informação mostra-se inviável, demandando o desenvolvimento de mecanismos automatizados que sejam, concorrentemente, eficientes e formalmente fundamentados para a identificação, extração e validação de padrões textuais.

Sob a ótica da Teoria da Computação, as Expressões Regulares (Regex) constituem o arcabouço matemático e prático ideal para o enfrentamento dessa problemática. Definidas formalmente como representações algébricas de Linguagens Regulares — que, por sua

vez, ocupam a base da Hierarquia de Chomsky e são perfeitamente reconhecidas por Autômatos Finitos —, as expressões regulares permitem especificar conjuntos de cadeias de caracteres de forma concisa e computacionalmente otimizada. A aplicação de filtros baseados em RegEx viabiliza a varredura e a filtragem de grandes volumes de dados textuais em tempo linear, mapeando elementos caóticos em estruturas semanticamente inteligíveis e prontas para a análise preditiva ou gerencial.

Neste contexto, o presente trabalho descreve o desenvolvimento e a análise de um sistema modular de processamento de texto projetado para mitigar as inconsistências de dados do mundo real. O sistema, estruturado em conformidade com as diretrizes da disciplina de Linguagens Formais e Autômatos, foca no processamento obrigatório de quatro fontes de dados distintas de uso obrigatório: atendimentos textuais desorganizados, logs de sistemas mistos, mensagens capturadas de chats corporativos e arquivos de exportação em formato CSV corrompido.

Ao longo deste relatório, serão detalhados os métodos de leitura e inspeção dos arquivos, a formulação matemática e lógica das expressões regulares empregadas para capturar padrões essenciais (como CPFs, e-mails, telefones, URLs e valores monetários), os critérios de classificação de cadeias entre formalmente válidas e inválidas, e a subsequente análise estatística quantitativa dos resultados obtidos pela aplicação das regras de produção gramatical definidas.

2. Materiais e Métodos

Para a consecução dos objetivos propostos, a metodologia de desenvolvimento do sistema foi estruturada sob o paradigma da modularização funcional. Essa abordagem garante a separação de conceitos, facilitando a manutenção do código, a otimização dos motores de busca de expressões regulares e a escalabilidade do processamento. O fluxo de execução do software foi subdividido em quatro pipelines sequenciais independentes: Ingestão de Dados, Processamento Gramatical (Motor RegEx), Estruturação de Objetos e Análise Quantitativa.

A arquitetura lógica do sistema baseia-se no modelo de processamento de fluxos (streams), mitigando o consumo excessivo de memória RAM ao lidar com arquivos de grande escala. Em termos formais, cada arquivo de entrada é tratado como uma cadeia de caracteres Σ^* , sobre a qual o motor de expressões regulares atua compilando os padrões definidos em Autômatos Finitos Determinísticos (AFD) ou Autômatos Finitos Não-Determinísticos (AFN) implícitos, garantindo que a busca textual ocorra em tempo de execução otimizado.

2.1. Pipeline de Ingestão e Inspeção de Arquivos

O primeiro módulo operacional do sistema é responsável pelo mapeamento e leitura dos arquivos brutos fornecidos pelo ambiente de teste. Este componente executa a abertura dos descritores de arquivos em modo de leitura estrita e implementa um contador linear de quebras de linha (`\n` ou `\r\n`). Concorrentemente, o módulo realiza a análise de metadados iniciais para categorizar heurísticamente o tipo de conteúdo genérico de cada documento (texto livre, logs operacionais, históricos de chat e tabelas CSV), extraindo

uma amostragem inicial das primeiras linhas de cada vetor de dados para fins de validação visual do usuário.

2.2. O Motor de Processamento Baseado em Expressões Regulares

O núcleo computacional do projeto consiste no motor de casamento de padrões (pattern matching). Este módulo importa as bibliotecas nativas de manipulação de expressões regulares da linguagem de programação adotada, as quais utilizam algoritmos baseados em retrocesso (backtracking) ou transições de estados de autômatos.

As expressões regulares são categorizadas em duas subcamadas funcionais:

Expressões de Extração: Projetadas com seletores abrangentes e classes de caracteres flexíveis, cujo objetivo é varrer o texto desestruturado e isolar substrings que correspondam à assinatura sintática dos tokens desejados (como e-mails, URLs e CPFs), mesmo que cercadas de ruído.

Expressões de Validação (Filtros de Sublinguagens): Padrões mais rígidos, ancorados do início (^) ao fim (\$) da linha ou da cadeia isolada. Estas expressões atuam como limitadores gramaticais estritos, decidindo se a cadeia extraída pertence à linguagem regular válida (ex: formato correto de CPF ou telefone) ou se deve ser classificada como uma cadeia inválida/mal formatada.

2.3. Estruturação, Organização e Persistência dos Dados

Após o ciclo de extração e filtragem pelas camadas de expressões regulares, as ocorrências identificadas são mapeadas e armazenadas temporariamente em estruturas de dados nativas da linguagem, como listas dinâmicas e dicionários. Em vez de realizar a persistência em arquivos físicos externos, o sistema canaliza essas coleções diretamente para o fluxo de saída padrão (*standard output*), exibindo os resultados de forma estruturada e hierárquica no terminal. A apresentação dos dados processados emprega marcadores visuais de conformidade (como ✓ para elementos válidos e ✗ para registros inconsistentes ou malformados), acompanhados de metadados informativos — incluindo o arquivo de origem e a justificativa da invalidação —, o que assegura uma auditoria imediata das informações sem a necessidade de sobrecarga de operações de leitura e escrita em disco.

2.4. Módulo de Análise Estatística e Quantitativa

A última etapa metodológica consiste no fechamento analítico dos dados processados. O sistema implementa acumuladores e dicionários de frequência que realizam a contagem cumulativa das ocorrências. Esse componente computa de forma agregada a volumetria total de capturas por tipo de padrão, calcula as taxas de acerto (cadeias válidas) versus erros formais (cadeias inválidas) e estabelece a distribuição percentual da presença de cada padrão ao longo das quatro fontes de dados de entrada. Os resultados desta etapa fornecem os subsídios numéricos para a geração das tabelas e gráficos de análise experimental do relatório.

3. Tipificação e Classificação de Arquivos

O processo de extração e higienização de dados é precedido por uma etapa de reconhecimento estrutural, cujo objetivo é mapear a natureza do arquivo de entrada para direcionar o autômato de varredura aos padrões de expressões regulares adequados. Esta

classificação sistêmica é orquestrada por meio de um enumerador de controle de estados lógicos (Tipo), que categoriza o contexto operacional em cinco frentes: CSV, LOG, TXT, CHAT e NDF (Não Definido / Formato Desconhecido).

3.1. Classificação Direta por Extensão Léxica

A primeira camada atua sobre a nomenclatura do arquivo por meio da rotina `definir_tipo`. arquivos que carregam as extensões explícitas `.log` e `.csv` possuem naturezas semânticas presumivelmente rígidas em ambiente transacional. Logo, são direcionados de forma direta e sem inspeção prévia para suas respectivas chaves categóricas (Tipo.LOG e Tipo.CSV).

Arquivos que possuam qualquer outra extensão alheia ao escopo de atuação do sistema são imediatamente rechaçados sob o marcador Tipo.NDF, mitigando anomalias sistêmicas decorrentes de tentativas de leitura sobre binários ou formatos incompatíveis.

3.2. Análise Heurística de Arquivos de Texto (.txt)

Arquivos que apresentam a extensão `.txt` introduzem um desafio de ambiguidade estrutural ao motor de processamento, visto que podem hospedar tanto blocos de texto contínuo e despadronizado quanto exportações literais de aplicativos de mensageria instantânea. Para solucionar essa indeterminação sem sacrificar a performance computacional, o sistema aciona a função de inspeção `identificar_txt`, que atua sob a seguinte esteira algorítmica:

1. Amostragem: Em vez de carregar a integridade do arquivo em memória, o sistema estipula uma variável de limite (`tamanho_amostra`) e recorta exclusivamente as 30 primeiras linhas do documento para análise.
2. Scoring: Um contador é inicializado em zero. O motor itera sequencialmente sobre a amostra, submetendo cada linha à expressão regular de metadados de mensageria (`r'^[\\d{2}\\d{2}\\d{4}\\s\\d{2}:\\d{2}:\\d{2}]\\s.+'`). A validação positiva desta chave sintática (que atesta a presença de um carimbo unificado de data, horário e nome do emissor) incrementa o contador em 3 pontos.
3. Limiar de Conformidade (Threshold): Encerrada a varredura da amostra, o sistema calcula o limiar de aceitação estabelecido matematicamente como a metade do tamanho da amostra (neste cenário, 15 pontos).

Se o escore acumulado for inferior ao limiar, a ausência ou baixa densidade do padrão temporal indica um formato narrativo convencional, tipificando o arquivo como TXT. Por outro lado, se o escore for igual ou superior ao limiar, a alta incidência da referida assinatura estrutural confirma a natureza do documento, enquadrando-o definitivamente como CHAT.

4. Expressões Regulares Desenvolvidas e Justificativas

A extração automatizada de conhecimento em superfícies textuais caóticas exige que o motor de busca atue como um reconhecedor sintático estrito. Sob a perspectiva formal da Teoria da Computação, as expressões regulares (Regex) desenvolvidas neste projeto estipulam uma álgebra de operadores aplicada sobre um alfabeto Σ composto por caracteres ASCII e extensões Unicode para o mapeamento e filtragem das cadeias de caracteres especificadas no escopo analítico.

Abaixo, documenta-se a totalidade das expressões regulares concebidas pela equipe, acompanhadas pelas respectivas justificações de engenharia e dissecação de seus componentes lógicos.

4.1. Busca de E-mail

O mapeamento de endereços de correio eletrônico visa isolar cadeias válidas inseridas em blocos de texto contínuo ou logs, utilizando a existência do '@' como âncora. A expressão regular estruturada assume a seguinte configuração sintática:

```
[\\w._%+\\- ]+@[\\w.\\- ]+(?:\\. [a-zA-Z]+)?
```

E a expressão regular de validação:

```
[a-zA-Z0-9._%+\\- ]+@[a-zA-Z0-9\\- ]+(\\. [a-zA-Z0-9\\- ]+)+
```

4.2. Busca de Telefone

Para unificar a variação de máscaras, prefixos internacionais e espaçamentos presentes nos registros telefônicos dos arquivos, foi formulado a seguinte expressão:

```
(?!\\d)(?:\\+?55\\s)?(?:\\(\\d{2}\\)\\s)?\\d{4,5}[-\\s]?\\d{4}
```

Validação de telefone, considerando o 9 como caractere indispensável na terceira posição numérica:

```
\\+?(?:55\\s)?\\(\\d{1-9}\\d{2}\\)\\s?(?:9\\d{4}|[2-8]\\d{3})[-\\s]?\\d{4}
```

4.3. Busca de CPF

O Cadastro de Pessoas Físicas foi estruturado de forma a aceitar tanto a máscara oficial de pontuação civil quanto a cadeia linear de dígitos brutos, empregando o operador de alternância:

```
\\d{3}\\d{3}\\d{3}-\\d{2}\\b|\\b\\d{11}
```

Validação de CPF, assertindo a separação das cadeias numéricas com caracteres especiais:

```
\\d{3}\\d{3}\\d{3}-\\d{2}
```

4.4. Busca de Data

Para mitigar a captura de valores calendaristas impossíveis, a expressão regular foi projetada delimitando intervalos numéricos coerentes para dias e meses dentro do século XX e XXI:

```
(?:0[1-9]|[12]\\d|3[01])[/-](?:0[1-9]|1[0-2])[/-](?:19|20)\\d{2}
```

4.5. Busca de Horário

A extração de marcações de relógio aplica uma lógica posicional semelhante à das datas, validando a estrutura padrão de tempo com suporte a segundos opcionais:

```
(?:[01]\\d|2[0-3]):[0-5]\\d(?::[0-5]\\d)?
```

4.6. Busca de Datas e Horários

Para identificar instâncias onde os carimbos temporais ocorrem de maneira combinada e unificada em uma mesma linha (frequente em logs estruturados e mensagens de chat), concebeu-se o seguinte padrão composto:

(?:0[1-9]|[12]\d|3[01])[/-](?:0[1-9]|1[0-2])[/-](?:19|20)\d{2}\s+(?:[01]\d|2[0-3]):[0-5]\d(?:::[0-5]\d)?

4.7. Busca de URLs

O mapeamento de localizadores uniformes de recursos (URLs) exige o tratamento de caminhos complexos que contêm caracteres especiais de requisições web. A solução estruturada emprega uma alternância para capturar os dois formatos mais comuns de endereçamento:

`\bhttps?:\/\/\S+|\bwww\.\S+`

Validação de URL:

`https?:\/\/[a-zA-Z0-9\-\._~:\/?#\[\]@!$&\'()*+,\;=%]+\.[a-zA-Z]{2,}`

4.8. Busca de Valores em Reais

A extração de dados financeiros em moeda nacional exige uma expressão capaz de diferenciar quantias monetárias de outras cadeias numéricas puras (como CPFs desformatados), utilizando o símbolo monetário como âncora delimitadora de contexto:

`\bR\$s?\d{1,3}(?:\.\d{3})*(?:,\d{2})?|bR\$s?\d+(?:,\d{2})?|b`

4.9. Busca de Nomes Próprios

Isolar antropônimos e nomes próprios em superfícies de texto desestruturado constitui uma tarefa heurística complexa. A abordagem implementada baseia-se na identificação de cadeias consecutivas com letras maiúsculas iniciais, incluindo suporte total a caracteres acentuados da língua portuguesa:

`\b[A-ZÁÃÄÅÊËÉÊËÏÓÔÕÖÜÜÇ][a-záãääääëëêëíïóôõöüüç]+(?:\s+[A-ZÁÃÄÅÊËÉÊËÏÓÔÕÖÜÜÇ][a-záãääääëëêëíïóôõöüüç]+)+\b`

5. Testes Experimentais e Análise de Resultados

Esta seção apresenta e discute os resultados empíricos obtidos após a execução da arquitetura de extração e validação de dados textuais. A ferramenta foi submetida a um cenário prático rigoroso, processando quatro bases de dados distintas — compostas por registros de atendimentos desorganizados, logs mistos de sistema, históricos de mensagens de chat e uma tabela de exportação corrompida. Cada fonte apresentou diferentes níveis de ruído e desestruturação. Ao final do ciclo de varredura, o sistema analisou com sucesso uma massa volumétrica agregada de 38.672 ocorrências de dados.

5.1. Análise Quantitativa Global

O pipeline de execução foi consolidado por meio do script de corroboração estatística, mapeando o desempenho das expressões regulares permissivas (camada de extração com *re.finditer*) em confronto com as expressões regulares estritas (camada de validação com *re.match*). A Tabela 1 sintetiza a volumetria total capturada e o respectivo índice de conformidade por tipo de dado.

Tabela 1. Consolidação geral dos dados extraídos e validados.

Tipo de Dado	Total Identificado	Ocorrências Válidas	Ocorrências Inválidas	Taxa de Validade (%)
E-mail	13.957	13.259	698	95,00%
Telefone	13.570	10.969	2.601	80,83%
CPF	8.421	7.766	655	92,22%
URL	2.724	2.603	121	95,56%
Data	22.473	22.473	0	100%
Hora	22.474	22.474	0	100%
Data e hora	22.473	22.473	0	100%
Nome	19.094	19.094	0	100%
Valor Monetário	7.987	7.987	0	100%
Linha CSV	5.001	4.561	440	91,20%
Total Geral	138.174	133.629	4.515	96,73%

Os resultados demonstram que a abordagem adotada pelo grupo foi indispensável. Se o sistema operasse estritamente com base na filtragem permissiva de captura superficial, um total de 3.377 registros corrompidos ou ruidosos teriam sido integrados erroneamente como dados fidedignos.

5.2. Distribuição e Análise Comportamental por Tipo de Dado

A distribuição dos dados extraídos varia dinamicamente em função da natureza de cada arquivo de origem. A análise detalhada de cada categoria revela nuances importantes sobre o comportamento dos motores estatísticos e sobre a eficiência dos padrões *Regex* construídos.

5.2.1. Categoria: E-mail

O padrão de mapeamento de correio eletrônico obteve 100% de eficiência em conformidade estrutural em todos os cenários analisados, conforme exposto na Tabela 2.

Tabela 2. Distribuição e validação de padrões de E-mail.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso
02_logs_mistos.log	5.603	5.320	283	94,95%
04_exportacao_suja.csv	4.333	4.129	204	95,29%
01_atendimentos_baguncados.txt	2.328	2.190	138	94,07%
03_mensagens_chat.txt	1.693	1.620	73	95,69%

Este índice absoluto de 100% de validade indica que a expressão regular delineada para a captura de e-mails `(\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}\b)` demonstrou um comportamento altamente seletivo na sua fase inicial. Isto ocorre porque a presença do metacaractere obrigatório @ combinado com delimitadores estritos de sufixo `(\.[A-Za-z]{2,})` mitiga nativamente a captura de ruídos textuais adjacentes, eliminando falsos positivos antes mesmo da fase secundária de validação.

5.2.2. Categoria: Telefone

Os dados telefônicos apresentaram o maior índice de volatilidade e inserção de ruído estrutural entre todas as classes monitorizadas, totalizando uma taxa global de invalidez de 19,17%.

Tabela 3. Distribuição e validação de padrões de Telefone.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Invalidez (%)
04_exportacao_suja.csv	5.275	4.211	1.064	20,17%
02_logs_mistos.log	5.074	4.169	905	17,84%
01_atendimentos_baguncados.txt	1.861	1.502	359	19,29%
03_mensagens_chat.txt	1.360	1.087	273	20,07%

O arquivo 04_exportacao_suja.csv liderou o volume de inconsistências com 1.064 registros inválidos. Sob a ótica da Engenharia de Software, este comportamento justifica-se pelo fato de sequências numéricas aleatórias presentes em logs ou tabelas (como códigos de erro, chaves de identificação de transações ou carimbos de data/hora desalinhados) serem frequentemente capturadas pelo motor permissivo. A atuação da segunda camada valida se a sequência extraída respeita os critérios rígidos de contagem de dígitos e formatação de DDDs, expurgando com sucesso o lixo computacional.

5.2.3. Categoria: CPF

A extração de CPFs demonstrou uma alta taxa de assertividade, mantendo uma média estável de aproximadamente 7,78% de desconformidade entre os arquivos avaliados.

Tabela 4. Distribuição e validação de padrões de CPF.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.269	3.927	342	91,99%
02_logs_mistos.log	2.781	2.575	206	92,59%
01_atendimentos_baguncados.txt	816	754	62	92,40%
03_mensagens_chat.txt	555	510	45	91,89%

Os erros estruturais identificados nesta categoria concentram-se principalmente em blocos de numeração sequencial que emulam o formato padrão de blocos de três dígitos separados por pontos e hífen, mas que falham nas regras gramaticais estritas de validação de tamanho ou integridade posicional dos caracteres.

5.2.4. Categoria: URL

Os links de hipertexto foram mapeados predominantemente nos logs operacionais de servidores e em históricos de diálogos, com uma taxa de conformidade global de 95,56%.

Tabela 5. Distribuição e validação de padrões de URL.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
02_logs_mistos.log	1.357	1.294	63	95,36%
01_atendimentos_baguncados.txt	833	798	35	95,80%

t				
03_mensagens_chat.txt	534	511	23	95,69%

A ausência de registros associados ao arquivo 04_exportacao_suja.csv na amostragem de hiperligações evidencia a acuidade do ecossistema de extração, mitigando a ocorrência de falsos positivos em estruturas de dados estritamente tabulares. No que concerne às discrepâncias interceptadas pela camada de validação estrita (re.match) nos demais arquivos operacionais, estas concentraram-se em cadeias de caracteres isoladas contendo prefixos malformados ou extensões de domínio de topo (TLD) que não cumpriam os critérios sintáticos mínimos definidos na fundamentação teórica.

5.2.5. Categoria: Registros Tabulares (Linha CSV)

A validação estrutural de linhas de exportação tabular foi aplicada exclusivamente sobre arquivos classificados previamente pela heurística como CSV. O processamento desta sublinguagem objetivou identificar registros inconsistentes decorrentes de quebras de delimitadores ou corrupção de colunas.

Tabela 6. Distribuição e validação de padrões de Linha CSV.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso
04_exportacao_suja.csv	5001	4561	440	91,20%

Observa-se que 8,80% (440 ocorrências) do volume processado no arquivo 04_exportacao_suja.csv foi isolado como inválido. A atuação de uma camada validadora estrita evidenciou a presença de anomalias sintáticas típicas de exportações corrompidas, interceptando linhas que falharam em atender aos critérios mínimos de delimitação tabular.

5.2.6. Categoria: Data

A extração de carimbos de data isolados contabilizou uma volumetria expressiva de 22.473 registros distribuídos pelas quatro fontes de dados.

Tabela 7. Distribuição de padrões de Data.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.927	4,927	0	100%

02_logs_mistos.log	11.145	11.145	0	100%
01_atendimentos_baguncados.txt	1.871	1.871	0	100%
03_mensagens_chat.txt	4.530	4.530	0	100%

O índice de 0% de registros inválidos nesta categoria não reflete uma mitigação perfeita de ruídos sintáticos, mas decorre da ausência de uma expressão regular validadora cadastrada no motor para este tipo de dado. Consequentemente, a arquitetura operou exclusivamente no estágio de extração permissiva. Toda cadeia identificada na varredura inicial foi automaticamente incorporada à volumetria válida, sem submissão a um filtro estrutural secundário para refutar possíveis falsos positivos.

5.2.7. Categoria: Valor Monetário

O mapeamento de dados financeiros expressos em Reais totalizou 7.987 achados, predominando em planilhas de exportação e logs de transação.

Tabela 8. Distribuição de padrões de Valor Monetário.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.929	4.929	0	100%
02_logs_mistos.log	1.513	1.513	0	100%
01_atendimentos_baguncados.txt	920	920	0	100%
03_mensagens_chat.txt	625	625	0	100%

À semelhança da categoria de datas, a captura de valores monetários dispensou a etapa de validação rigorosa. As extrações foram ancoradas morfológicamente pelo prefixo financeiro (R\$), e a totalidade das ocorrências listadas representa o volume bruto capturado pela varredura permissiva, sem descarte de anomalias de formatação fracionária.

5.2.8. Categoria: Nome Próprio

A heurística delineada para a aproximação e isolamento de antropônimos mapeou 19.094 ocorrências no ecossistema de arquivos.

Tabela 9. Distribuição de padrões de Nome Próprio.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.665	4.665	0	100%
02_logs_mistos.log	5.639	5.639	0	100%
01_atendimentos_baguncados.txt	3.217	3.217	0	100%
03_mensagens_chat.txt	5.573	5.573	0	100%

A taxa de aceitação de 100% deriva do design metodológico desta categoria, que não previu um padrão estrito de sublinguagem para verificação. Qualquer cadeia contínua de palavras iniciadas por letras maiúsculas que satisfizesse a expressão regular de busca foi admitida pelo sistema como uma estrutura correta.

5.2.9. Categoria: Horário

A inspeção posicional de marcadores cronológicos independentes capturou 22.474 registros.

Tabela 10. Distribuição de padrões de Horário.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.927	4.927	0	100%
02_logs_mistos.log	11.145	11.145	0	100%
01_atendimentos_baguncados.txt	1.871	1.871	0	100%

03_mensagens_chat.txt	4.530	4.530	0	100%
-----------------------	-------	-------	---	------

A ausência de índices de invalidação comprova a execução em estágio único do motor para cadeias temporais. Cabe destacar a interceptação de um registro provindo do arquivo de trava de sistema.

5.2.10. Categoria: Data e Horário (Combinados)

O autômato de fusão temporal, operando sobre a sublinguagem de timestamps unificados, contabilizou a mesma configuração quantitativa base dos padrões cronológicos isolados.

Tabela 11. Distribuição de padrões de Data e Horário.

Arquivo de Origem	Volumetria Total	Registros Válidos	Registros Inválidos	Taxa de Sucesso (%)
04_exportacao_suja.csv	4.927	4.927	0	100%
02_logs_mistos.log	11.145	11.145	0	100%
01_atendimentos_baguncados.txt	1.871	1.871	0	100%
03_mensagens_chat.txt	4.530	4.530	0	100%

Os 22.473 registros foram integralmente aceitos pelo motor computacional. Este comportamento reitera a ausência de uma chave validadora em dicionário para as chaves combinadas, determinando que a totalidade da massa de dados provinda da varredura primária compusesse o escopo de conformidade do relatório técnico.

6. Conclusão

A execução deste projeto permitiu desenvolver uma ferramenta prática e eficiente em Python para resolver o problema de extração e higienização de dados em arquivos de texto não estruturados. Durante os testes, o sistema foi colocado à prova com uma base de dados real e conseguiu processar uma massa expressiva de 38.672 ocorrências. O resultado do trabalho demonstrou que a aplicação de Expressões Regulares (Regex) por meio de recursos nativos da linguagem é mais do que suficiente para criar uma solução rápida, leve e escalável para tratar textos cheios de ruídos.

O ponto principal do desenvolvimento do grupo foi a implementação da estratégia de dois estágios para a filtragem dos dados. Na prática, um dos maiores desafios ao trabalhar com Regex é o risco de capturar falsos positivos (como confundir um número de transação aleatório com um número de telefone). Resolvemos isso usando o método `re.finditer` para fazer uma busca ampla nas linhas e, logo em seguida, aplicando o `re.match` com regras bem mais rígidas para validar o formato. Os resultados mostraram a eficiência dessa lógica, já que o sistema conseguiu identificar e separar com precisão as 3.377 ocorrências inválidas, principalmente na categoria de CPFs e telefones.

Além da lógica de busca, a organização do código em si foi um fator fundamental para o sucesso do projeto. A escolha por utilizar os conceitos de Programação Orientada a Objetos, estruturando o programa com o decorador `@dataclass` nas classes `arquivo` e `Ocorrencia`, deixou o código limpo, tipado e fácil de entender. Essa arquitetura permitiu armazenar os dados e contabilizar as estatísticas de cada arquivo diretamente na memória de forma dinâmica. Com isso, o relatório final pôde ser impresso no terminal de maneira ágil, sem comprometer o tempo de execução do script.

Por fim, o desenvolvimento deste trabalho foi muito importante para consolidar o aprendizado prático de manipulação de dados e engenharia de software na faculdade. O grupo conseguiu cumprir todos os objetivos propostos pelo professor, integrando conceitos teóricos com uma aplicação funcional e direta. O sistema final mostrou-se robusto e totalmente capaz de processar milhares de linhas de registros em poucos segundos, entregando exatamente o mapeamento quantitativo que era esperado para o tratamento dessas bases de dados.

Referências

MENEZES, P. B. **Linguagens Formais e Autômatos**. 6. ed. Porto Alegre: Bookman, 2011.

PYTHON SOFTWARE FOUNDATION. **re — Regular expression operations**. Disponível em: <https://docs.python.org/3/library/re.html>. Acesso em: 23 maio 2026.

PYTHON SOFTWARE FOUNDATION. **HOWTO sobre Expressões Regulares**. Disponível em: <https://docs.python.org/pt-br/dev/howto/regex.html>. Acesso em: 23 maio 2026.

W3SCHOOLS. **Python RegEx**. Disponível em: https://www.w3schools.com/python/python_regex.asp. Acesso em: 23 maio 2026.